RetailPulse

AI-Powered Customer Analytics & Demand Forecasting Platform

Customer Segmentation • Churn Risk Scoring • Demand Forecasting • Inventory Optimization

Prepared by: Shiney Priyanka R

B.Tech CSBS, S A Engineering College, Chennai

Data Science & Analytics Internship — Zidio Development

July 2026

Dataset: Brazilian E-Commerce Public Dataset by Olist (Kaggle)

1. Project Overview

RetailPulse is an end-to-end retail analytics platform built on the Brazilian E-Commerce Public Dataset by Olist (Kaggle), covering 110,189 order-line records across 93,350 unique customers. The project transforms raw multi-table transactional data into actionable business intelligence spanning sales performance, customer segmentation, churn risk, demand forecasting, and inventory optimization — surfaced through both a live Streamlit web dashboard and a six-page Power BI report.

Vision & Objectives

- Understand customer purchasing behavior through RFM-based segmentation
- Identify churn-prone customers and quantify revenue at risk
- Forecast short-term product demand to support planning decisions
- Classify inventory by revenue contribution (ABC analysis) and calculate reorder quantities (EOQ)• Present all of the above through interactive, filterable dashboards

Target Users / Use Cases

- E-commerce operations and business analysts monitoring sales health
- Customer retention teams identifying at-risk segments
- Inventory / supply chain planners prioritizing stock attention

Business Value Delivered

- Quantified BRL 7.61M in revenue tied to Medium/High churn-risk customers — a concrete target forretention campaigns
- Identified that just 28.2% of products (Class A) drive 80% of total revenue — focusing inventory effortwhere it matters most
- Delivered a working 8-week revenue forecast (BRL 267,608 average weekly) for planning purposes

Non-Functional Goals

- Lean, reproducible dependency footprint (4 core packages for the deployed app)
- Fast dashboard load time suitable for free-tier cloud hosting
- Modular pipeline: each analytical stage (EDA, RFM, churn, forecasting, inventory) is an independent,re-runnable notebook

2. Key Features

ID	Feature	Description	Result
F1	Data Pipeline	Cleans & merges 9 raw Olist CSVs into one master dataset	110,189 rows, 93,350 unique customers
F2	Exploratory Data Analysis	Revenue, category, and time-trend analysis	BRL 15.42M total revenue; health_beauty top category; Nov 2017 peak
F3	RFM Customer Segment	ationRecency-Frequency-Monetary scoring nto 8 segments	Loyal Customers 25.1%, Champions 12.5%
F4	Churn Risk Scoring	RFM-based Low/Medium/High risk tiering	25.1% High Risk; BRL 7.61M revenue at risk (Med+High)
F5	Demand Forecasting	Compared Naive baseline vs. LightGBM on weekly revenue	Naive baseline won (WAPE 16.1% vs. 25.1%)
F6	Inventory Optimization	ABC classification + Economic Order Quantity (EOQ) for 31,619 products	Class A (28.2% of products) drives 80% of revenue
F7	Interactive Dashboards	4-page Streamlit web app + 6-page Power BI report	Live public demo; synced slicers & drill-through in Power BI

3. Technology Stack

Category	Technology	Rationale
Language	Python	Core data processing and modeling
Data Processing	pandas, numpy	Cleaning, merging, feature engineering across 9 source tables
Machine Learning	scikit-learn, LightGBM	Churn risk scoring; demand forecast benchmarking
Visualization (Web)	Streamlit, Plotly	Interactive, publicly deployable dashboard
Visualization (BI)	Power BI, DAX	Semantic data model, calculated measures, cross-filtered multi-page report
Version Control	Git & GitHub	Source control, collaboration-ready history, submission delivery
Deployment	Streamlit Community Cloud	Free public HTTPS hosting, auto-redeploy on GitHub push

Note on scope: This project deliberately uses a lean, reproducible stack sized to the dataset and timeline. Forecast model selection was evidence-based: a Naive baseline was retained over LightGBM after weekly WAPE comparison showed it performed better on this dataset — a genuine finding rather than a default assumption that more complexity means better accuracy.

4. Architecture Overview

RetailPulse follows a linear, notebook-driven analytics pipeline. Each stage reads from and writes to a shared `data/processed/` directory, so any stage can be re-run independently without re-running the whole pipeline.

1	**Raw Sales Data**	9 Olist CSVs (orders, items, customers, payments, reviews, products, sellers, geolocation)
2	**Data Pipeline**	src/pipeline.py — cleaning, merging, feature engineering -> master.csv (110,189 rows)
3	**EDA**	Revenue, category & time-trend analysis
4	**RFM Segmentation**	8 customer segments by Recency/Frequency/Monetary score
5	**Churn Risk Scoring**	Low/Medium/High risk tiers derived from RFM
6	**Demand Forecasting**	Naive baseline vs. LightGBM comparison (WAPE)
7	**Inventory Optimization**	ABC classification + EOQ for 31,619 products
8	**Dashboards**	Streamlit (4 pages, live) + Power BI (6 pages, DAX measures, synced slicers)

5. Execution Approach

Work progressed in five natural phases, moving from raw data to a fully deployed, dual-dashboard product:

Phase 1 — Data Foundation

Sourced the Olist dataset from Kaggle, built the cleaning/merge pipeline, and produced the unified master dataset used by every downstream stage.

Phase 2 — Exploratory & Customer Analysis

Ran EDA to establish baseline revenue/category/time patterns, then built RFM segmentation and churn risk scoring on top of those patterns.

Phase 3 — Forecasting & Inventory

Benchmarked Naive vs. LightGBM demand forecasts and built ABC/EOQ inventory optimization logic.

Phase 4 — Dashboard Development

Built the 4-page Streamlit dashboard reading directly from processed CSVs, then built a parallel 6-page Power BI report with a proper relational data model and DAX measures.

Phase 5 — Deployment & Documentation

Pushed the repository to GitHub, deployed the Streamlit app to Streamlit Community Cloud (resolving dependency and path-configuration issues along the way), and compiled this documentation.

6. Technical Highlights

Model Performance

Metric	Result
Demand Forecast — Naive Baseline WAPE	16.1%
Demand Forecast — LightGBM WAPE	25.1% (underperformed Naive)
Avg Weekly Revenue Forecast	BRL 267,608
8-Week Total Revenue Forecast	BRL 2,140,861
Churn Risk Segmentation	High 25.1% / Medium 24.7% / Low 50.2%
Revenue at Risk (Medium + High)	BRL 7.61M (49.4% of total revenue)
Inventory ABC Split	Class A: 28.2% of products -> 80% of revenue

Feature Engineering

- RFM scores: recency (days since last purchase), frequency (order count), monetary (total spend)
- Time-based features: order month, week, year extracted for seasonality analysis
- Category-level and product-level revenue rollups for ABC classification

Challenges Faced & Solutions

Power BI relationship errors: Initial many-to-many cardinality conflicts between master and inventory_abc were resolved by relating on the correct unique key (product_id) rather than the repeating category field.

Broken percentage measure: A 'Class A Revenue %' DAX measure produced nonsensical values (>1000%) when sliced by ABC class, caused by an unintended filter context on the denominator. Fixed using REMOVEFILTERS() to force the correct total.

Empty requirements.txt on deployment: Streamlit Cloud failed with ModuleNotFoundError because the initial requirements.txt was generated from the wrong Python environment. Rebuilt it by cross-checking actual imports against the correct environment's installed packages.

Hardcoded local file path: A leftover os.chdir() call pointing to a local Windows path crashed the app on Streamlit Cloud's Linux servers. Removed it, relying on relative paths that already worked correctly from the repo root.

7. Deployment & Operations

Deployment Platform

- Streamlit Community Cloud — public web hosting for the live dashboard
- GitHub — version control and source of truth for Streamlit Cloud's auto-deploy
- Power BI Desktop — local report authoring with a proper star-schema-style relational model

Deployment Workflow

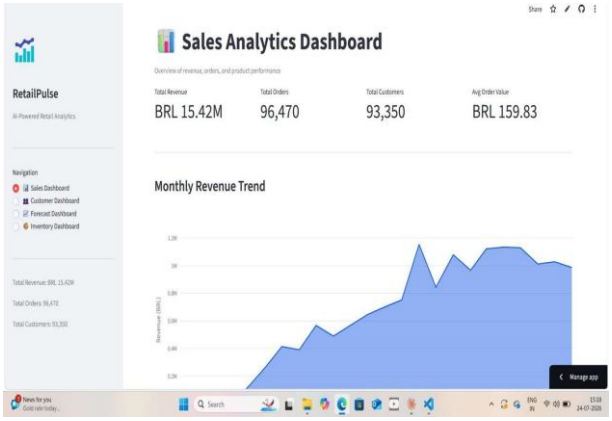
- Code and processed data pushed to a public GitHub repository (raw source CSVs excluded via .gitignore to keep the repo lightweight)
- Streamlit Community Cloud connected directly to the GitHub repo; every push to main triggers an automatic redeploy. Dependencies pinned in requirements.txt to the four packages actually required at runtime (streamlit, pandas, numpy, plotly)

Reproducibility

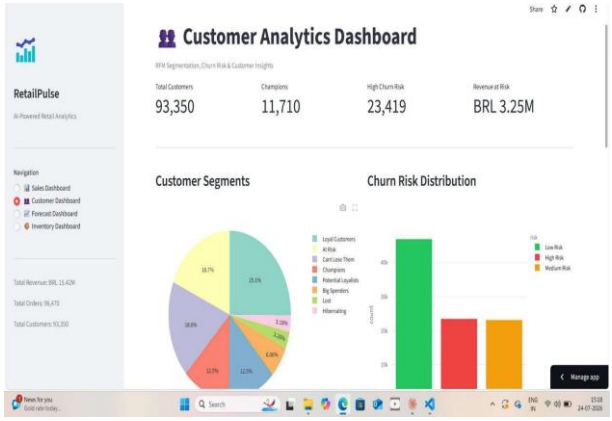
The repository separates raw data, processed data, source pipeline code, notebooks, and the dashboard application into clearly named folders, with setup instructions in README.md covering dependency installation and how to regenerate processed data from the raw Kaggle dataset.

8. Visuals

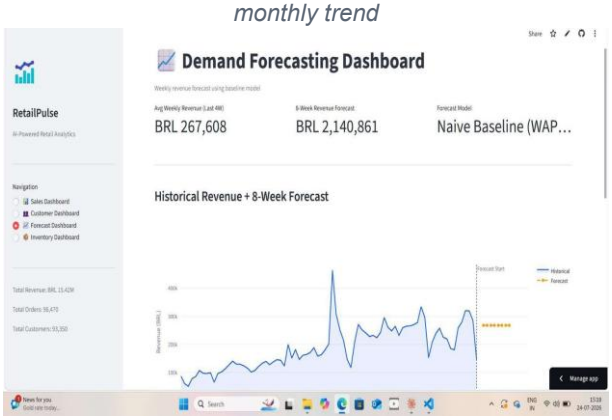
Streamlit Dashboard — Live Application



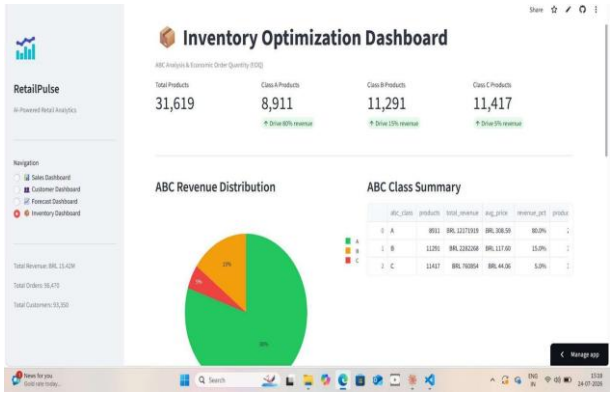
Sales Analytics Dashboard — revenue, orders & monthly trend



Customer Analytics Dashboard — RFM segments & churn risk

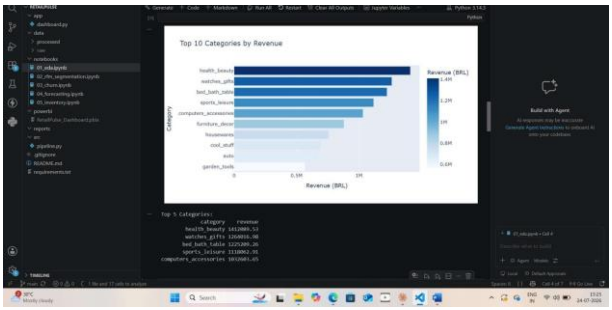


Demand Forecasting Dashboard — 8-week revenue forecast



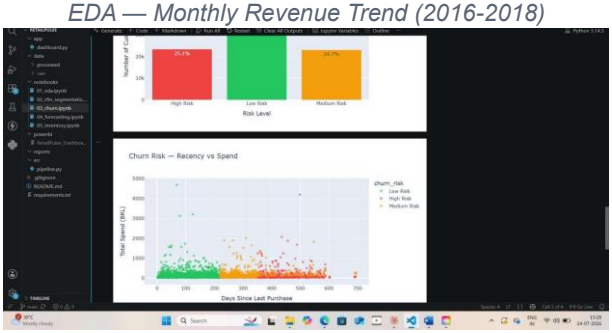
Inventory Optimization Dashboard — ABC revenue distribution

Notebook Outputs — Analysis & Modeling

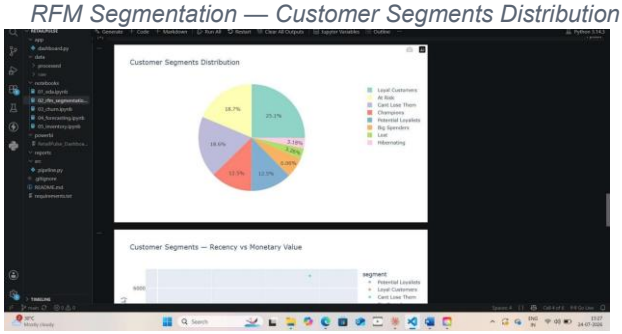


EDA — Monthly Revenue Trend (2016-2018)

RFM Segmentation — Customer Segments Distribution

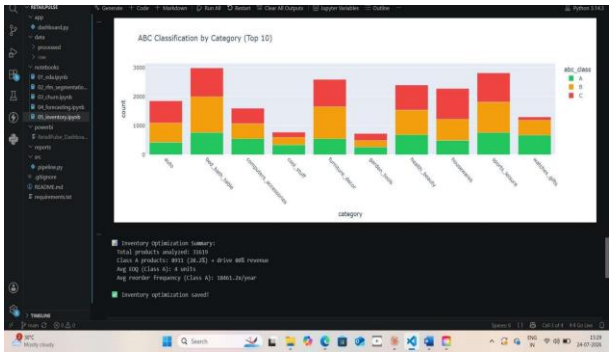


Churn Risk — Recency vs. Spend by Risk Tier



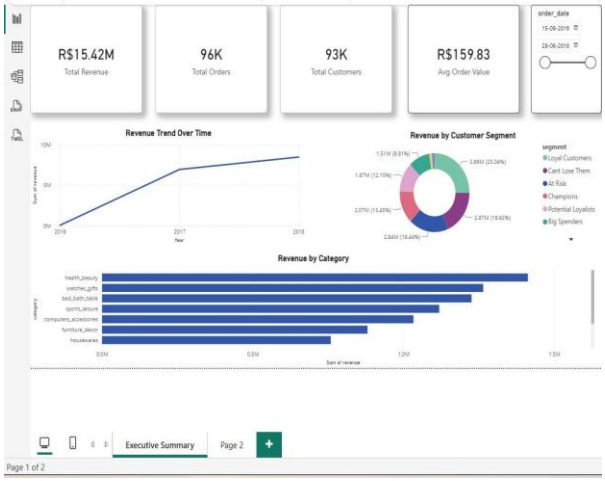
Forecasting — Weekly Revenue Forecast vs. Actual

(Naive wins)

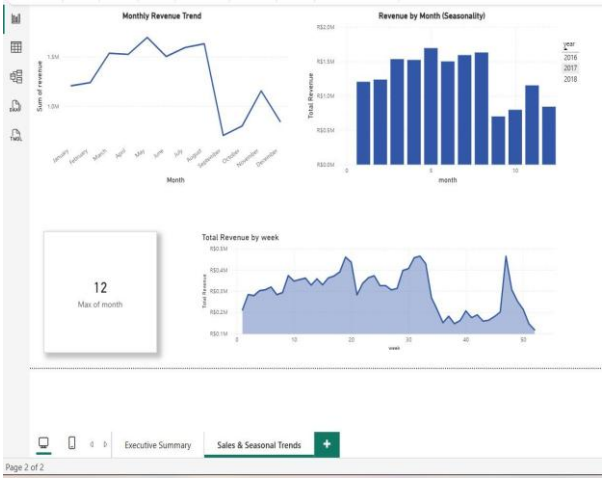


Inventory — ABC classification chart

Power BI Report — 6-Page Interactive Dashboard

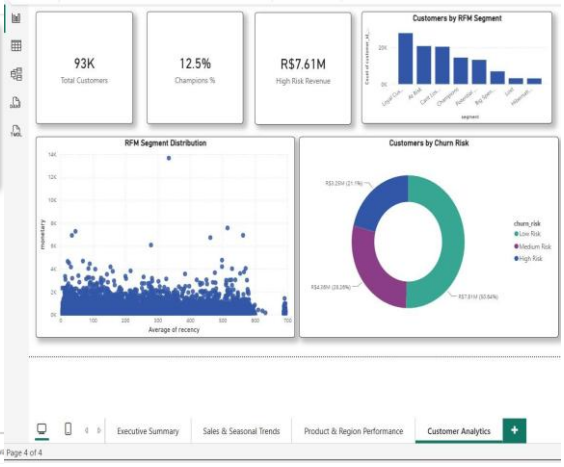


Executive Summary — revenue, orders & segment mix

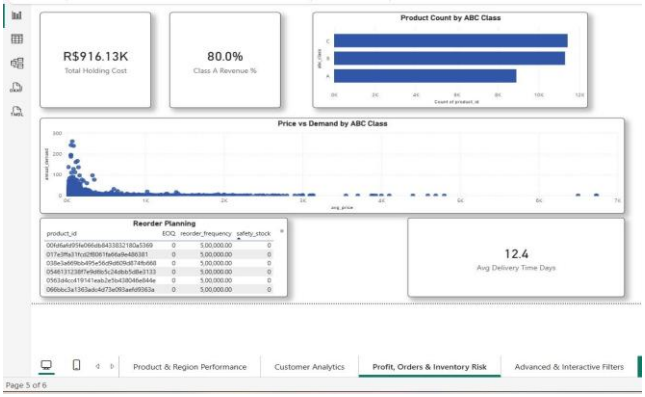


Sales & Seasonal Trends — monthly seasonality pattern

Product & Region Performance — revenue by state



Customer Analytics — RFM distribution & churn breakdown



Profit, Orders & Inventory Risk — ABC class & delivery time

The full 6-page Power BI report additionally includes an Advanced & Interactive Filters page with synced slicers (date range, category, customer segment) across all pages, plus a category-level drill-through page — demonstrated live in the accompanying demo video.

9. Personal Reflection

This project gave me hands-on experience with the full analytics lifecycle — from messy, multi-table raw data through to a deployed, interactive product used by real end users. A few things stood out.

Key Learnings

- Validating model choice with real metrics matters more than assuming complexity equals accuracy —the Naive-vs-LightGBM forecasting result was a genuine, useful surprise
- Debugging a Power BI semantic model taught me how relationship cardinality and filter context cansilently produce wrong numbers that still look plausible until checked
- Deploying to Streamlit Cloud exposed the real gap between 'works on my machine' and areproducible, portable deployment — particularly around relative file paths and pinned dependencies

Industry Best Practices Applied

- Modular, re-runnable pipeline stages rather than one monolithic script
- Version control with a clean .gitignore excluding raw data and large binaries
- Minimal, verified dependency pinning instead of a full environment dump

Future Roadmap Ideas

- True delivery-delay tracking (requires an estimated-delivery-date field not present in this dataset)
- Segment-specific forecasting rather than one aggregate model
- Automated data refresh scheduling for the Power BI report

"Building RetailPulse taught me that good analytics is as much about questioning your own model's results as it is about building the model in the first place."